Predicting Billboard #1 Hits

Authors: May Eindra Tet Toe, Anastasia Tountas, Tran Anh Thu Phung, Harry Nguyen

Summary

Insert a summary of your analysis here.

Introduction

Insert an introduction to your analysis here.

Methods and Results

Overview of our Methodology

The Billboard Hot 100 dataset provides us with two datasets: `billboard.csv`, which contains one row per song that reached #1 along with it's features, and `topics.csv`, a reference list of lyrical theme labels (`lyrical_topics`). Our team decided to combine the two datasets to `billboard_clean` in order to create more comprehensive data to better equip our model. Here is an overview of our data analysis methodology:

1. **Reading & Wrangling:** read and wrangle the dataset into one tidy combined dataset
2. **Exploratory Data Analysis:** explore the data to understand the distribution of features and their relationships with the target variable
3. **Model Building:** build a regression model to predict the number of weeks a song stays at #1 based on its features
4. **Model Evaluation:** evaluate the performance of our model using appropriate metrics and validation techniques
5. **Results and Conclusion:** summarize our findings, discuss the implications of our results, and suggest potential future work or improvements to our analysis

Load libraries and set seed

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.4.3
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.2     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ---------------------------------------- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(tidytuesdayR)
```

```
## Warning: package 'tidytuesdayR' was built under R version 4.4.3
```

```r
library(knitr)
library(dplyr)
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
library(tidymodels)
```

```
## Warning: package 'tidymodels' was built under R version 4.4.3
```

```
## -- Attaching packages --------------------------------------- tidymodels 1.4.1 --
## v broom        1.0.12     v rsample      1.3.2
## v dials        1.4.2      v tailor       0.1.0
## v infer        1.1.0      v tune         2.0.1
## v modeldata    1.5.1      v workflows    1.3.0
## v parsnip      1.4.1      v workflowsets 1.1.1
## v recipes      1.3.1      v yardstick    1.3.2
```

```
## Warning: package 'broom' was built under R version 4.4.3
```

```
## Warning: package 'dials' was built under R version 4.4.3
```

```
## Warning: package 'scales' was built under R version 4.4.3
```

```
## Warning: package 'infer' was built under R version 4.4.3
```

```
## Warning: package 'modeldata' was built under R version 4.4.3
```

```
## Warning: package 'parsnip' was built under R version 4.4.3
```

```
## Warning: package 'recipes' was built under R version 4.4.3
```

```
## Warning: package 'rsample' was built under R version 4.4.3
```

```
## Warning: package 'tailor' was built under R version 4.4.3
```

```
## Warning: package 'tune' was built under R version 4.4.3
```

```
## Warning: package 'workflows' was built under R version 4.4.3
```

```
## Warning: package 'workflowsets' was built under R version 4.4.3
```

```
## Warning: package 'yardstick' was built under R version 4.4.3
```

```
## -- Conflicts ------------------------------------------ tidymodels_conflicts() --
## x rsample::calibration()   masks caret::calibration()
## x scales::discard()        masks purrr::discard()
## x dplyr::filter()          masks stats::filter()
## x recipes::fixed()         masks stringr::fixed()
## x dplyr::lag()             masks stats::lag()
## x caret::lift()            masks purrr::lift()
## x yardstick::precision()   masks caret::precision()
## x yardstick::recall()      masks caret::recall()
```

```
## x yardstick::sensitivity() masks caret::sensitivity()
## x yardstick::spec()        masks readr::spec()
## x yardstick::specificity() masks caret::specificity()
## x recipes::step()          masks stats::step()
```

```
set.seed(123)
```

Reading & Wrangling our dataset from the web into R

- The two datasets were joined using a `left_join()` on `lyrical_topic` (billboard) and `lyrical_topics`(topics).
- **Identifier columns** (`song`, `artist`, `date`) were removed as they are not predictive features.
- **`non_consecutive`** was removed to prevent data leakage — it can only be determined after a song has completed its full chart run.
- **`rating_1, rating_2, rating_3`** were removed in favour of `overall_rating` (their mean) to avoid multicollinearity.
- **Free-text columns** were removed avoid the high dimensionality and noise associated with Natural Language Processing: `lyrics`, `u_s_artwork`, `sound_effects`, `song_structure`, `featured_artists`, `songwriters`, `songwriters_w_o_interpolation_sample_credits`, `producers`, `double_a_side`, and `talent_contestant`.
- **High-cardinality columns** removed because they generalise poorly: `label`, `parent_label`, and `artist_place_of_origin`.
- **Redundant columns** were removed: `keys` is replaced by `simplified_key`; `cdr_style` and `discogs_style` are replaced by `cdr_genre` and `discogs_genre` respectively.
- **`discogs_genre`** was removed in favour of `cdr_genre` because cdr has more reliable class definitions, thus improving internal validity.
- **Ambiguous columns** were removed: `featured_in_a_then_contemporary_play`, `featured_in_a_then_contemporar` and `featured_in_a_then_contemporary_t_v_show`.
- All remaining character columns were converted to factors using `as.factor()`, and rows with any missing values were dropped with `drop_na()`.

```
# Load the dataset
tuesdata <- tidytuesdayR::tt_load(2025, week = 34)
```

```
## ---- Compiling #TidyTuesday Information for 2025-08-26 ----
## --- There are 2 files available ---
##
##
## -- Downloading files --------------------------------------------------------
##
##   1 of 2: "billboard.csv"
##   2 of 2: "topics.csv"
```

```
# Extract the datasets
billboard <- tuesdata$billboard
topics <- tuesdata$topics
```

```
# Join the two datasets
billboard_joined <- billboard |>
  left_join(topics, by = c("lyrical_topic" = "lyrical_topics"))
```

```
billboard_clean <- billboard_joined |>
  # Remove identifier columns
  select(-c(song, artist, date)) |>
  # Remove data-leakage column
  select(-non_consecutive) |>
  # Remove individual judge scores to avoid multicollinearity
```

```
  select(-c(rating_1, rating_2, rating_3)) |>
  # Remove free-text columns
  select(-c(
    lyrics,
    u_s_artwork,
    sound_effects,
    song_structure,
    featured_artists,
    songwriters,
    songwriters_w_o_interpolation_sample_credits,
    producers,
    double_a_side,
    talent_contestant
  )) |>
  # Remove high-cardinality columns
  select(-c(label, parent_label, artist_place_of_origin)) |>
  # Remove redundant columns and discogs_genre
  select(-c(cdr_style, discogs_style, keys, discogs_genre)) |>
  # Remove ambiguous columns
  select(-c(
    featured_in_a_then_contemporary_play,
    featured_in_a_then_contemporary_film,
    featured_in_a_then_contemporary_t_v_show
  )) |>
  # Convert remaining character columns to factors
  mutate(across(where(is.character), as.factor)) |>
  drop_na()
```

Exploratory Data Analysis

First, we find the summary statistics for our target variable, `weeks_at_number_one`, and our key numeric predictors: `overall_rating`, `bpm`, `energy`, `danceability`, `happiness`, `acousticness`, `loudness_d_b`, `front_person_age`, and `length_sec`.

```
## Warning in attr(x, "align"): 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
## See help("Deprecated")
```

```
## Warning in attr(x, "format"): 'xfun::attr()' is deprecated.
## Use 'xfun::attr2()' instead.
## See help("Deprecated")
```

Table 1: Summary statistics

Variable	Mean	SD	Min	Median	Max
acousticness	31.51	27.66	0	23.00	98
bpm	115.50	23.69	16	115.00	176
danceability	62.45	15.21	14	64.00	98
energy	59.81	19.62	3	60.50	98
front_person_age	27.97	6.51	11	27.50	62
happiness	62.60	24.96	4	68.00	99
length_sec	222.88	50.12	96	224.00	515
loudness_d_b	-9.02	3.40	-23	-9.00	-1
overall_rating	6.18	1.86	1	6.33	10

Variable	Mean	SD	Min	Median	Max
weeks_at_number_one	2.91	2.49	1	2.00	16

Then, we examine the distribution of our target variable, `weeks_at_number_one`, to check for skewness.

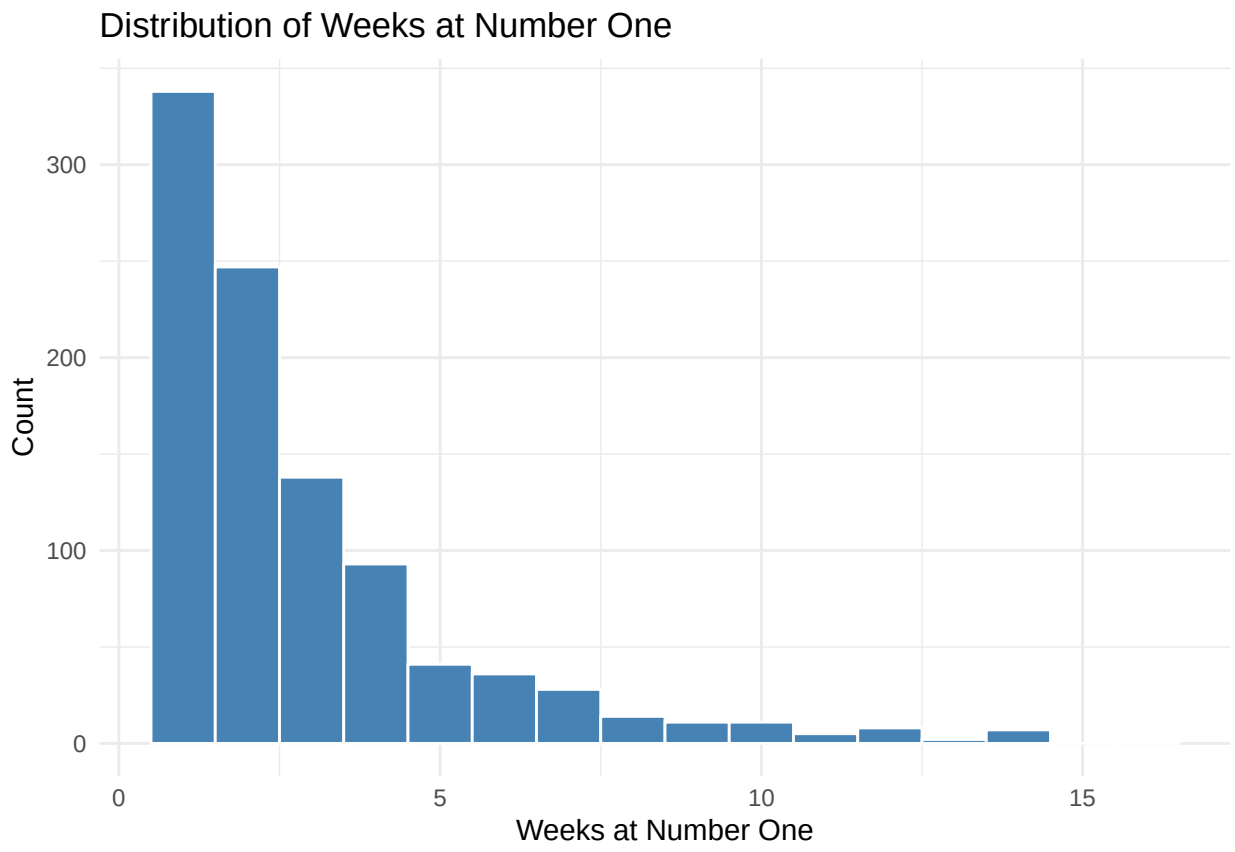

Figure 1: Distribution of weeks at number one.

The distribution is heavily right-skewed. The majority of songs spent only 1 to 3 weeks at #1, with counts dropping off sharply afterwards. A small number of songs stayed for 10 or more weeks, forming a long right tail.

Next, we examine the distributions of the five Spotify audio features, to identify any skewness that may require transformation before modelling.

`acousticness` is heavily right-skewed, with most songs clustered near zero. `happiness` shows a bimodal pattern with peaks around 30 and around 80. `energy` is very slightly left-skewed, while `bpm` and `danceability` are roughly normal.

Finally, we compare average `weeks_at_number_one` by genre to see whether certain genres are associated with longer chart runs.

Electronic/Dance;Latin seems to have the highest average (~10 weeks). Pop;Reggae and Folk/Country;March also rank highly. Mainstream genres like Pop and Rock sit in the middle of the range. At the lower end, we can find niche genres like Electronic/Dance;Funk/Soul, averaging to 1 week.

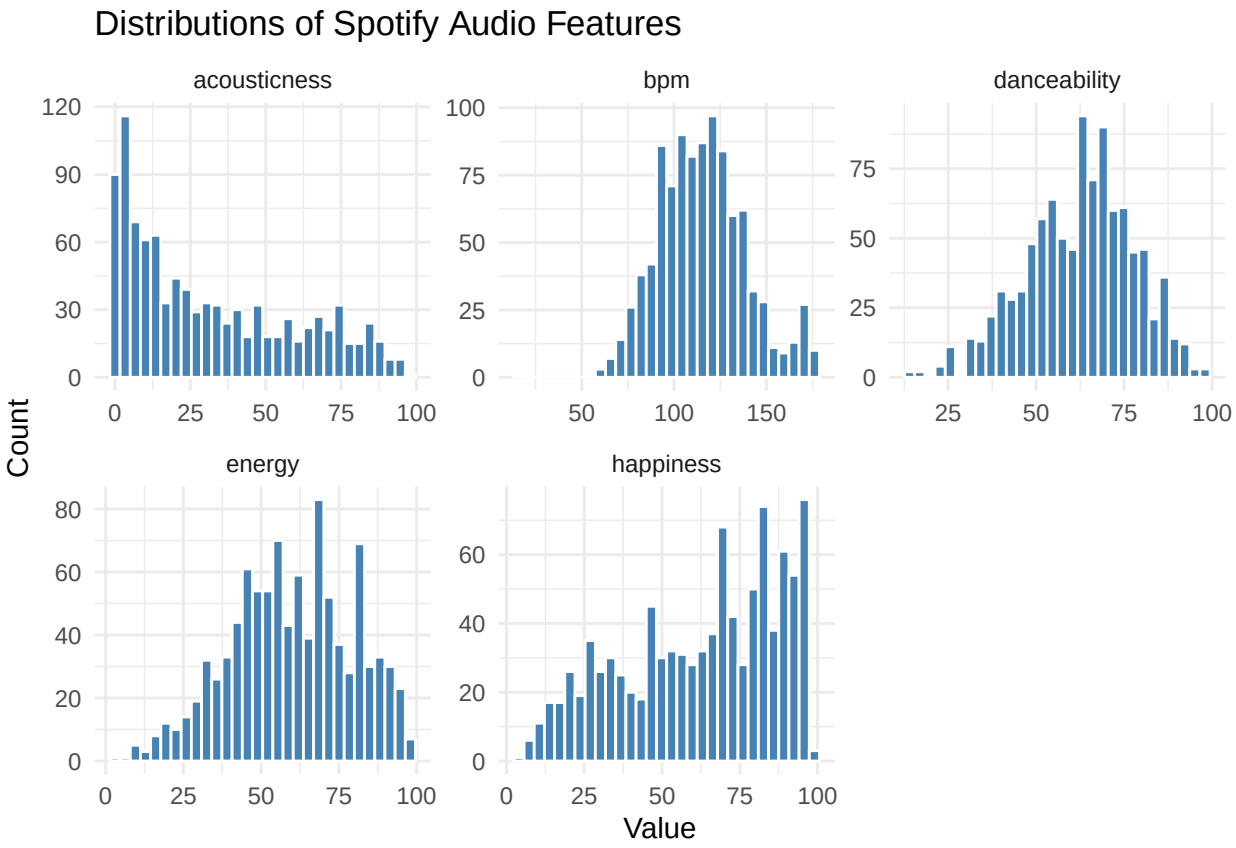

Figure 2: Distributions of the five Spotify audio features.

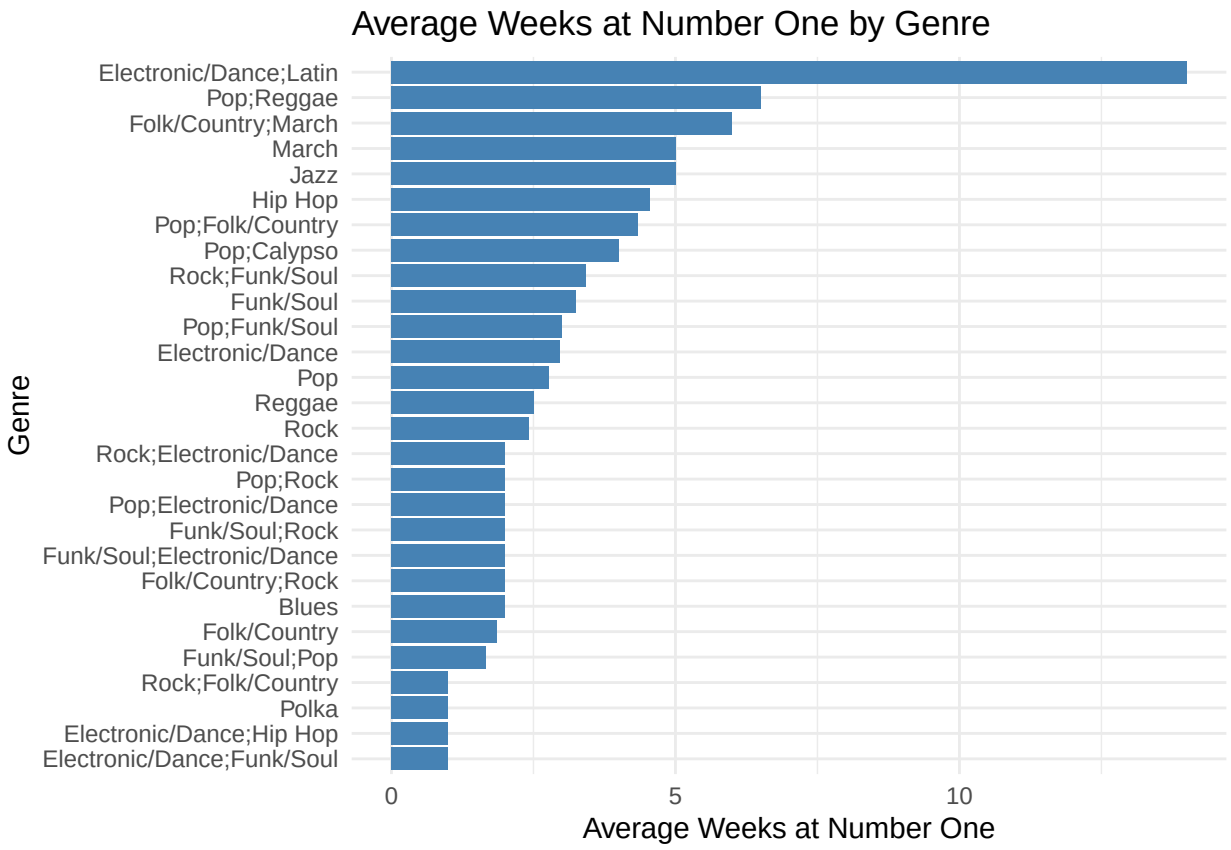

Figure 3: Average weeks at number one by genre.

Feature Transformations

Based on the Exploratory Data Analysis, two variables require transformation before modelling:

- **weeks_at_number_one** is heavily right-skewed
- **acousticness** is also heavily right-skewed

Therefore, we apply log transformation to these three variables to compresses the tail and produce a more symmetric target distribution. We will be using `log1p()` instead of `log()` to handle any zero values in the data.

```
billboard_log_transformed <- billboard_clean |>
  mutate(
    log_weeks_at_number_one = log1p(weeks_at_number_one),
    log_acousticness = log1p(acousticness)
  )
```

Moreover, several songs in the dataset were associated with more than one musical genre (for example, `Electronic/Dance; Latin` or `Pop;Funk/Soul`). Treating these combined labels as single categorical values would incorrectly imply that each combination represents a distinct genre and would lead to sparse, poorly interpretable factor levels. To address this, the `cdr_gerne` variable was processed as a multi-label feature. Individual genre were first separated into their component parts and then encoded as binary indicator variables using one-hot encoding.

```
billboard_genres <- billboard_log_transformed |>
  separate_rows(cdr_genre, sep = ";") |>
  mutate (cdr_genre = str_trim(cdr_genre))

billboard_genres_wide <- billboard_genres |>
  mutate(value = 1) |>
  pivot_wider(
    names_from = cdr_genre,
    values_from = value,
    values_fill = 0
  )
```

The `simplified_key` variable originally contained 25 levels representing individual musical keys in standard notation (e.g. `C`, `Am`, `Bbm`). Retaining all 25 levels would generate an excessive number of dummy variable during modelling, introducing sparsity and increasing the risk of overfitting. To address this, the keys were collapsed into three musically meaningful categories: Major (e.g. `C`, `Ab`, `F`), Minor (e.g. `Am`,`Bbm`, `Em`), and Multiple Keys.

```
billboard_simplify_key <- billboard_genres_wide |>
  mutate(
    simplified_key = case_when(
      simplified_key == "Multiple Keys"          ~ "Multiple Keys",
      str_ends(simplified_key, "m")              ~ "Minor",  # Am, Bbm, Bm, Cm, etc.
      TRUE                                        ~ "Major"  # A, Ab, B, Bb, C, etc.
    ) |> as.factor()
  )

billboard_model_final <- billboard_simplify_key |>
  select(-weeks_at_number_one, -acousticness)
```

Feature Selection

Drop Redundant Columns Columns were dropped on conceptual grounds across five categories:

- Demographic identity columns encoding the race and gender of artists, songwriters, and producers.
- Artist structure indicators, songwriter/producer role overlap columns.
- Timing-derived columns.
- Lyric redundant content.
- External content.

```
columns_to_drop <- c(
  # Demographic identity
  "artist_male","artist_white", "artist_black", "songwriter_male", "songwriter_white", "producer_male",
  # Artist structure
  "artist_structure", "group_named_after_non_lead_singer",
  # Songwriter/producer overlaps
  "artist_is_only_songwriter", "artist_is_only_producer", "songwriter_is_a_producer",
  # Timing
  "instrumental_length_sec",  "intro_length_sec",
  # Lyrical content
  "lyrical_topic","lyrical_narrative",
  # External content
  "written_for_a_play", "written_for_a_film", "written_for_a_t_v_show", "eurovision_entry", "topped_the_

billboard_manual_dropped <- billboard_model_final |>
  select(-all_of(columns_to_drop))
```

Drop columns with nearly zero variance : Near-zero variance predictors were identified and removed using `nearZeroVar()` from the `caret` package. A predictor was flagged for removal if it met either of two criteria: its most common value appeared at least 19 times more frequently than the second most common value (`freqCut = 95/5`), or fewer than 5% of its values were unique (`uniqueCut = 5`).

```
# Identify and remove near-zero variance predictors
nzv_cols <- nearZeroVar(billboard_manual_dropped,
                        freqCut = 95/5,
                        uniqueCut = 5,
                        names = TRUE)
nzv_cols
```

```
##  [1] "posthumous"                   "time_signature"
##  [3] "accordion"                    "banjo"
##  [5] "bongos"                       "clarinet"
##  [7] "cowbell"                      "flute_piccolo"
##  [9] "harmonica"                    "human_whistling"
## [11] "kazoo"                        "mandolin"
## [13] "pedal_lap_steel"              "ocarina"
## [15] "sitar"                        "trumpet"
## [17] "ukulele"                      "violin"
## [19] "rap_verse_in_a_non_rap_song"  "instrumental"
## [21] "free_time_vocal_introduction" "live"
## [23] "foreign_language"             "Folk/Country"
## [25] "March"                        "Jazz"
## [27] "Polka"                        "Reggae"
## [29] "Calypso"                      "Blues"
## [31] "Latin"
```

```
billboard_model_selected <- billboard_manual_dropped |>
  select(-all_of(nzv_cols))
```

Regression Model

Split the dataset into training and testing sets, with 705 of the observations allocated to the training set and 30% to the testing set.

```
# Train/test split
billboard_split <- initial_split(billboard_model_selected, prop = 0.7)
train_data <- training(billboard_split)
test_data <- testing(billboard_split)
```

A linear regression workflow was constructed using a preprocessing recipe that dummy-encoded categorical variables, removed near-zero variance and collinear predictors, and standardized numeric predictors. The model was fit using ordinary least squares on the training data.

```
lm_recipe <- recipe(log_weeks_at_number_one ~ ., data = train_data) |>
  step_dummy(all_nominal_predictors(), one_hot = FALSE) |>
  step_nzv(all_predictors()) |>
  step_normalize(all_numeric_predictors()) |>
  step_lincomb(all_numeric_predictors())

prep(lm_recipe) |> bake(new_data = NULL) |> glimpse()
```

```
## Rows: 686
## Columns: 38
## $ overall_rating               <dbl> 0.06012931, -0.11999606, -1.02062290, 0.7~
## $ divisiveness                 <dbl> 0.4989845, -1.5882310, 0.4989845, -0.1967~
## $ multiple_lead_vocalists      <dbl> -0.5286235, -0.5286235, -0.5286235, 1.888~
## $ front_person_age             <dbl> -0.29475019, 1.08748951, -1.36982551, 0.4~
## $ artist_is_a_songwriter       <dbl> 0.763835, 0.763835, -1.307275, 0.763835, ~
## $ artist_is_a_producer         <dbl> -0.725996, -0.725996, -0.725996, -0.72599~
## $ bpm                          <dbl> -0.1899626, -0.3157082, -0.1899626, -1.78~
## $ energy                       <dbl> 0.30537174, 1.29670806, 0.04449377, -0.79~
## $ danceability                 <dbl> 0.71918960, 0.51858074, -0.81881169, -0.6~
## $ happiness                    <dbl> 0.77203245, 0.73140851, -1.29978831, -1.3~
## $ loudness_d_b                 <dbl> 0.5937448, 0.8908337, 0.2966559, -1.78296~
## $ vocally_based                <dbl> -0.2419845, -0.2419845, -0.2419845, -0.24~
## $ bass_based                   <dbl> 2.018795, -0.494623, -0.494623, -0.494623~
## $ guitar_based                 <dbl> 0.822359, 0.822359, 0.822359, -1.214241, ~
## $ piano_keyboard_based         <dbl> 0.9453692, -1.0562458, 0.9453692, 0.94536~
## $ orchestral_strings           <dbl> -0.6660052, -0.6660052, 1.4993009, 1.4993~
## $ horns_winds                  <dbl> -0.6682854, 1.4941853, -0.6682854, -0.668~
## $ falsetto_vocal               <dbl> -0.4461056, -0.4461056, -0.4461056, -0.44~
## $ handclaps_snaps              <dbl> -0.3907918, 2.5551771, -0.3907918, -0.390~
## $ saxophone                    <dbl> -0.2519389, 3.9634297, -0.2519389, -0.251~
## $ length_sec                   <dbl> -0.16603410, 3.23164268, -1.47877286, 3.0~
## $ vocal_introduction           <dbl> -0.2771479, -0.2771479, -0.2771479, -0.27~
## $ fade_out                     <dbl> -1.2441568, 0.8025856, 0.8025856, -1.2441~
## $ cover                        <dbl> -0.4176158, -0.4176158, -0.4176158, -0.41~
## $ sample                       <dbl> -0.2740902, -0.2740902, -0.2740902, -0.27~
## $ interpolation                <dbl> -0.3341683, -0.3341683, -0.3341683, -0.33~
## $ inspired_by_a_different_song <dbl> -0.647825, -0.647825, -0.647825, -0.64782~
## $ spoken_word                  <dbl> -0.3036808, -0.3036808, -0.3036808, -0.30~
## $ explicit                     <dbl> -0.4484501, -0.4484501, -0.4484501, -0.44~
## $ log_acousticness             <dbl> -1.18809397, 0.66158181, -0.25132153, 0.0~
## $ Pop                          <dbl> -0.725996, -0.725996, -0.725996, 1.375410~
## $ Rock                         <dbl> 1.5913471, -0.6274824, 1.5913471, -0.6274~
```

```
## $ `Funk/Soul`                  <dbl> -0.5308781, 1.8809257, -0.5308781, -0.530~
## $ `Electronic/Dance`           <dbl> -0.2647649, 3.7714294, -0.2647649, -0.264~
## $ `Hip Hop`                    <dbl> -0.2950189, -0.2950189, -0.2950189, -0.29~
## $ log_weeks_at_number_one      <dbl> 0.6931472, 1.0986123, 1.0986123, 1.609437~
## $ simplified_key_Minor         <dbl> -0.584779, 1.707555, -0.584779, -0.584779~
## $ simplified_key_Multiple.Keys <dbl> -0.3177675, -0.3177675, -0.3177675, -0.31~
```

```
lm_spec <- linear_reg() |>
  set_engine("lm") |>
  set_mode("regression")

lm_workflow <- workflow() |>
  add_recipe(lm_recipe) |>
  add_model(lm_spec)
```

The linear regression workflow was evaluated using repeated 5-fold cross-validation, stratified by `log_weeks_at_number_one` , with RMSE, R^2 , and MAE used as performance metrics.

```
set.seed(123)
cv_folds <- vfold_cv(
  train_data,
  v       = 5,
  repeats = 3,
  strata  = log_weeks_at_number_one
)

cv_results <- fit_resamples(
  lm_workflow,
  resamples = cv_folds,
  metrics   = metric_set(rmse, rsq, mae),
  control   = control_resamples(save_pred = TRUE)
)

# Cross-validation summary
cv_metrics <- collect_metrics(cv_results)
print(cv_metrics)
```

```
## # A tibble: 3 x 6
##   .metric .estimator   mean     n std_err .config
##   <chr>   <chr>       <dbl> <int>   <dbl> <chr>
## 1 mae     standard   0.415     15 0.00599 pre0_mod0_post0
## 2 rmse    standard   0.505     15 0.00674 pre0_mod0_post0
## 3 rsq     standard   0.0626    15 0.0113  pre0_mod0_post0
```

Fit the model to the training data

```
final_fit <- fit(lm_workflow, data = train_data)
```

The fitted model was evaluated on the test set by comparing predicted and observed values using RMSE, R^2 , and MAE.

```
test_predictions <- predict(final_fit, new_data = test_data) |>
  bind_cols(test_data |> select(log_weeks_at_number_one)) |>
  rename(
    predicted = .pred,
    actual    = log_weeks_at_number_one
  )
```

```r
test_metrics <- test_predictions |>
  metrics(truth = actual, estimate = predicted)

print(test_metrics)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard      0.502
## 2 rsq     standard      0.0518
## 3 mae     standard      0.395
```

Results

Visualization Actual version predicted weeks at number one on the test set, shown on the original scale, with the red dashed line indicating perfect prediction.

```r
rsq_label <- paste0(
  "R_squared = ",
  round(
    test_predictions |>
      metrics(truth = actual, estimate = predicted) |>
      filter(.metric == "rsq") |>
      pull(.estimate),
    3
  )
)

p_actual_vs_predicted <- test_predictions |>
  mutate(
    actual_weeks    = expm1(actual),
    predicted_weeks = expm1(predicted)
  ) |>
  ggplot(aes(x = actual_weeks, y = predicted_weeks)) +
  geom_point(alpha = 0.4, colour = "steelblue", size = 2) +
  geom_abline(
    slope     = 1,
    intercept = 0,
    linetype  = "dashed",
    colour    = "firebrick",
    linewidth = 0.8
  ) +
  annotate(
    "text",
    x      = 12,
    y      = 2,
    label  = rsq_label,
    colour = "firebrick",
    size   = 4
  ) +
  labs(
    title    = "Actual vs Predicted Weeks at #1 (Test Set)",
    subtitle = "Dashed line = perfect prediction; points below = underestimate",
    x        = "Actual Weeks at #1",
```

```
    y        = "Predicted Weeks at #1"
  ) +
  theme_minimal()

print(p_actual_vs_predicted)
```

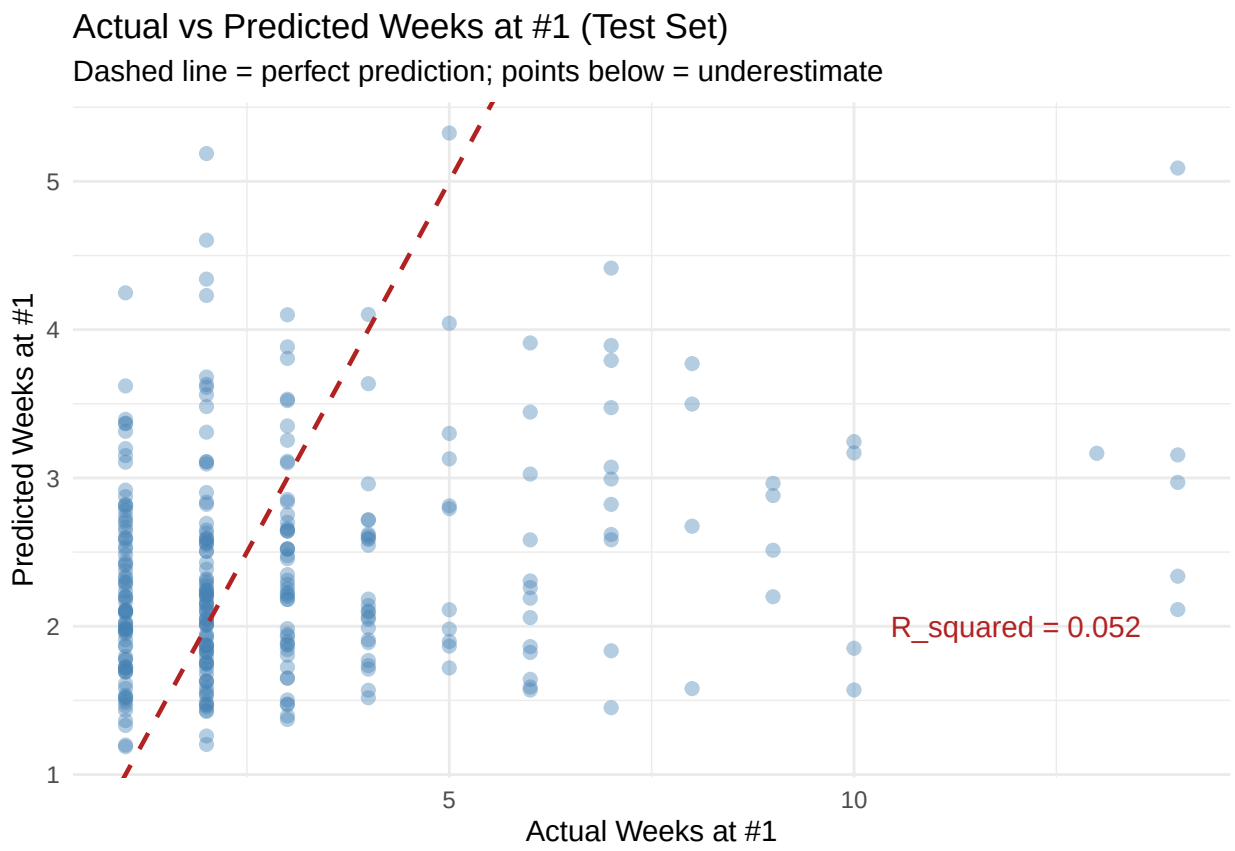

Figure 4: Actual vs predicted weeks at number one on the test set (original scale).

Top 15 predictors by standardized coefficient magnitude, with color indicating positive or negative effect on the response.

```
# Extract coefficients safely using broom::tidy explicitly
coef_data <- broom::tidy(extract_fit_engine(final_fit)) |>
  filter(term != "(Intercept)") |>
  arrange(desc(abs(estimate))) |>
  slice_head(n = 15) |>
  mutate(
    direction = if_else(estimate > 0, "Positive", "Negative"),
    term      = str_replace_all(term, "_", " ") |> str_to_title()
  )

p_coefficients <- ggplot(
  coef_data,
  aes(x = reorder(term, abs(estimate)), y = estimate, fill = direction)
) +
  geom_col(show.legend = TRUE) +
```

```
  coord_flip() +
  scale_fill_manual(
    values = c("Positive" = "steelblue", "Negative" = "firebrick"),
    name   = "Effect on\nlog(weeks + 1)"
  ) +
  labs(
    title    = "Top 15 Predictors by Coefficient Magnitude",
    subtitle = "Standardised coefficients - larger absolute value = stronger effect",
    x        = NULL,
    y        = "Standardised Coefficient"
  ) +
  theme_minimal() +
  theme(legend.position = "right")

print(p_coefficients)
```

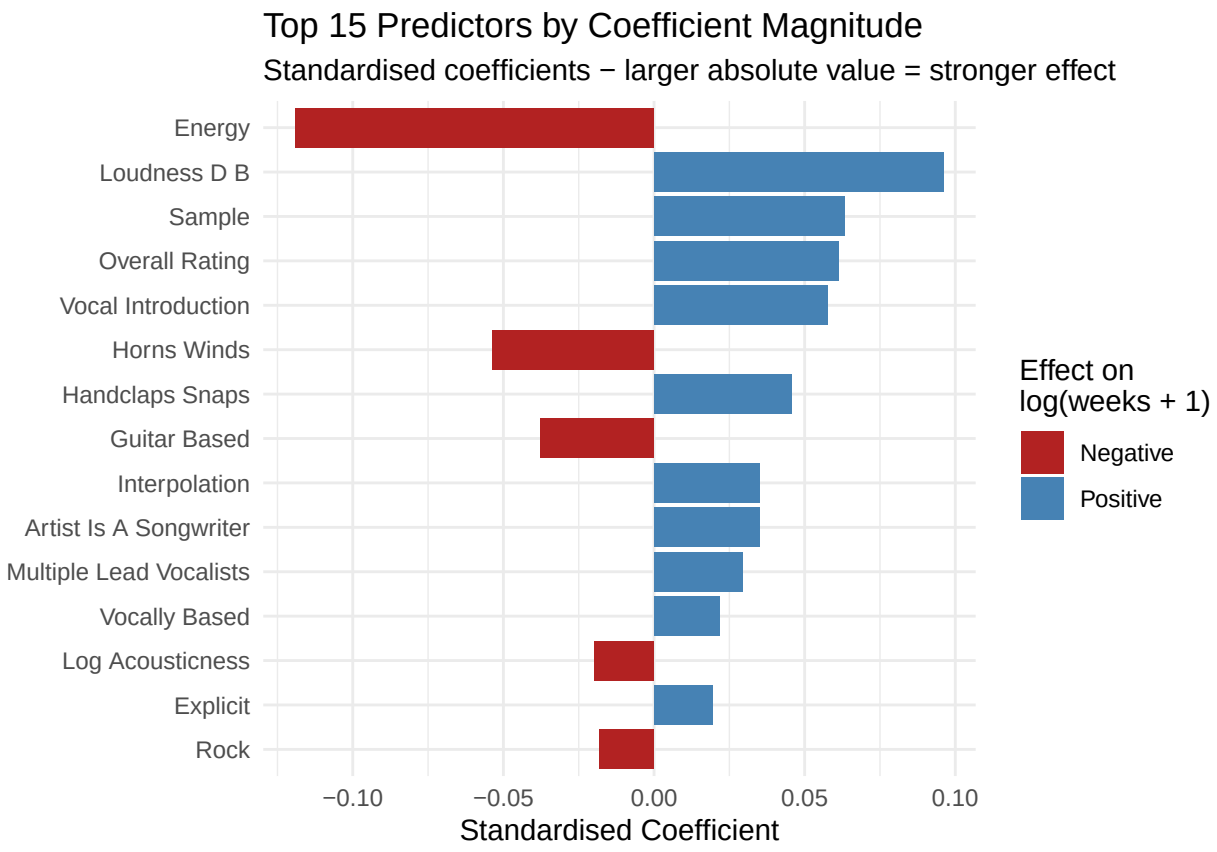

Figure 5: Top 15 predictors by standardised coefficient magnitude.